VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY

FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

Design and FPGA Implementation of a Winograd Convolution Accelerator for CNN Inference

Course Project Report

Course Name: Hardware Architectures for Machine Learning

Course Code: EE5223

Group Number: Group 07

Group Members:

Nguyen Minh Duc	*Student ID:* 2570662
Le The Anh	*Student ID:* 2570660

Instructor:

Truong Quang Vinh, Assoc. Prof. Ph.D

Faculty of Electrical and Electronics Engineering

Date of Submission: November 2026

Ho Chi Minh City, Vietnam

Abstract / Executive Summary

This project presents the design, mathematical verification, RTL implementation, and FPGA evaluation of a high-performance fixed-point hardware accelerator for Convolutional Neural Network (CNN) inference based on the **Winograd minimal filtering algorithm** $F(2 \times 2, 3 \times 3)$. In standard 2D convolution, generating a 2×2 output feature patch with a 3×3 kernel demands 36 multiplications. By exploiting Winograd's spatial domain transformation, the required multiplication count is reduced to 16, yielding a theoretical $2.25\times$ **arithmetic multiplication reduction**.

To realize this algorithmic efficiency on hardware within an intensive one-month engineering development cycle, the accelerator scope is constrained to a single-tile fixed-point architecture utilizing **signed INT16** precision. The complete datapath comprises a 4×4 input buffering unit, a multiplierless $B^T dB$ input transform unit, a parameterizable Winograd-domain element-wise multiply engine (configurable for 4, 8, or 16 parallel DSP blocks), an input-channel partial-sum accumulator, a multiplierless $A^T MA$ output transform unit, and an automated finite state machine (FSM) controller. Pre-transformed weights ($U = GgG^T$) are pre-computed offline to eliminate kernel transformation hardware from the critical path.

The verification strategy follows a strict hierarchical methodology: a floating-point Python model verified against direct convolution across $\geq 1,000$ test cases, an INT16 bit-accurate fixed-point software model, and self-checking SystemVerilog testbenches executing automated assertion-based checking against software-generated test vectors. Finally, the Winograd accelerator is synthesized and benchmarked against a fair direct-convolution hardware baseline targeting the same FPGA device and precision. Experimental results demonstrate that while Winograd significantly reduces DSP multiplier consumption and execution latency per tile, it introduces additional LUT/FF transform logic overhead and register buffering, establishing a definitive trade-off boundary for edge AI deployments.

Keywords: Winograd Algorithm, CNN Inference, FPGA Accelerator, SystemVerilog, Hardware Datapath, Fixed-Point Arithmetic, Digital IC Design, AMD Vivado / Intel Quartus.

Contents

Abstract / Executive Summary **i**

1 Introduction **1**
1.1 Project Background and Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Scope and Boundaries of the Report . . . 2

2 Theoretical Background and Mathematical Formulation **2**
2.1 Computing Paradigms and Accelerator Fundamentals . . . 2
2.2 The Winograd Minimal Filtering Algorithm $F(2 \times 2, 3 \times 3)$ . . . 3
2.2.1 Matrix Formulation . . . 3
2.2.2 Transformation Matrices . . . 4
2.3 Arithmetic Complexity Analysis . . . 4

3 Design Methodology and Verification Flow **4**
3.1 FPGA Design Flow . . . 4
3.2 FPGA Architecture vs. ASIC Standard-Cell Fabrication . . . 5
3.3 Hierarchical Verification Strategy . . . 5

4 Hardware Datapath Architecture and Evaluation Results **6**
4.1 Datapath Architecture Overview . . . 6
4.1.1 Stage 1: 4×4 Input Tile Buffer . . . 6
4.1.2 Stage 2: Input Transform Unit ($V = B^T dB$) . . . 7
4.1.3 Stage 3: Parameterizable Multiplier Engine . . . 7
4.1.4 Stage 4: Channel Accumulator . . . 7
4.1.5 Stage 5: Output Transform Unit ($Y = A^T MA$) . . . 8
4.2 Comparative Benchmarking Results . . . 8
4.3 Power and Energy Efficiency Analysis . . . 9
4.4 Application Case Studies . . . 9

5 Discussion and Engineering Insights **9**
5.1 Evaluation Against Success Criteria . . . 9
5.2 Technical Challenges and Mitigation Strategies . . . 10

6 Conclusion and Future Recommendations **10**
6.1 Conclusion . . . 10
6.2 Limitations and Future Work . . . 10

References **12**

Appendices **13**

A SystemVerilog Hardware Datapath Modules **13**

B Python Golden Model and Test Vector Exporter **13**

C Detailed Synthesis and Resource Utilization Logs **14**

List of Figures

1 Hierarchical Design and Verification Methodology Flow. 5
2 Block Diagram of the Pipelined Winograd Accelerator Core. 6

List of Tables

1 Comparison of FPGA Reconfigurable Flow and ASIC Manufacturing Flow. 6
2 Architectural Comparison: Direct 3x3 Baseline vs. Winograd $F(2 \times 2, 3 \times 3)$. 8
3 Detailed FPGA Device Resource Utilization Breakdown (Xilinx Artix-7 XC7A100T). 15

1 Introduction

1.1 Project Background and Context

Deep Convolutional Neural Networks (CNNs) have established themselves as the foundational backbone for modern artificial intelligence tasks, spanning autonomous driving perception, medical diagnostics, robotics, and mobile vision. However, CNN execution is heavily dominated by two-dimensional convolution operations, which commonly constitute over 85% to 90% of total computational workloads and memory accesses during inference.

As AI models are increasingly migrated from cloud datacenters to power-constrained edge platforms (e.g., smart cameras, edge gateways, and automotive Electronic Control Units), conventional general-purpose Central Processing Units (CPUs) and energy-hungry Graphics Processing Units (GPUs) struggle to meet tight real-time latency and thermal design power (TDP) constraints. Consequently, domain-specific hardware accelerators implemented on Field Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs) have emerged as the primary solution for achieving deterministic throughput with superior energy efficiency.

1.2 Problem Statement

Direct spatial convolution is fundamentally compute-bound and memory-bandwidth constrained. The standard sliding-window approach computes output activations via exhaustive Multiply-Accumulate (MAC) sequences. For small 3×3 kernels—which dominate modern network architectures such as VGG, ResNet, and MobileNet—direct convolution exhibits substantial arithmetic redundancy.

While fast algorithms such as Fast Fourier Transform (FFT) can reduce multiplication complexity for large filter kernels, FFT is notoriously inefficient for small kernels (e.g., 3×3) due to heavy complex-arithmetic transformation overheads and large memory footprint. Hence, the central engineering challenge is to minimize multiplication operations for small-kernel CNN inference without incurring prohibitive hardware complexity or unacceptable quantization error.

1.3 Objectives

The primary objectives of this project are:

1. **Mathematical Modeling:** Formulate and verify the Winograd $F(2 \times 2, 3 \times 3)$ minimal filtering algorithm against direct 3×3 convolution using Python, and establish an optimal signed INT16 fixed-point quantization scheme.

2. **RTL Architecture Design:** Design a modular, pipelined SystemVerilog hardware core comprising input transformation, parameterizable Winograd-domain multiplication (4, 8, or 16 DSP multipliers), channel accumulation, and output transformation.

3. **Automated Verification:** Construct a robust, self-checking simulation testbench hierarchy driven by software-generated vectors with automated pass/fail assertion reporting.

4. **FPGA Synthesis and Comparative Evaluation:** Synthesize and implement both the Winograd core and an architecturally fair direct-convolution baseline on the target FPGA, extracting and analyzing hardware utilization (LUT, FF, DSP, BRAM), maximum operating frequency ($F_{\max}$), latency (cycles/tile), throughput (GOPS), and numerical accuracy.

1.4 Scope and Boundaries of the Report

To guarantee rigorous completion within the strictly constrained one-month timeline, this project focuses on a verified accelerator core rather than a complete end-to-end multi-layer deployment:

- **Included Scope:** 2D 3×3 convolution, stride 1, valid padding, inference-only mode, signed INT16 arithmetic, pre-computed offline weight transformation ($U = GgG^T$), parameterizable datapath, and comparison against a direct 3×3 convolution core.

- **Explicitly Deferred Scope:** Full multi-layer network scheduling, external DDR memory controllers, AXI-DMA bus interfacing, arbitrary kernel dimensions ($1 \times 1, 5 \times 5$), multiple Winograd variants (e.g., $F(4 \times 4, 3 \times 3)$), and dynamic power measurement on physical silicon.

2 Theoretical Background and Mathematical Formulation

2.1 Computing Paradigms and Accelerator Fundamentals

Hardware architectures for deep learning inference span a continuum between flexibility and energy efficiency:

- **CPUs & GPUs (Temporal Architectures):** Utilize large register files and caches, executing instructions sequentially or in SIMD/SIMT warps. High flexibility but considerable instruction decoding and memory management power overhead.

- **NPUs / TPUs / ASICs (Spatial Architectures):** Utilize hardwired 2D systolic arrays or dedicated Processing Elements (PEs) with localized data reuse, maximizing MAC throughput per watt.
- **FPGAs (Reconfigurable Spatial Computing):** Combine spatial execution parallelism with post-silicon reconfigurability. Composed of configurable Look-Up Tables (LUTs), Flip-Flops (FFs), dedicated DSP48 slices, and distributed Block RAMs (BRAM), FPGAs are ideal for custom arithmetic pipelines such as Winograd transforms.

2.2 The Winograd Minimal Filtering Algorithm $F(2 \times 2, 3 \times 3)$

The minimal filtering theory developed by Shmuel Winograd establishes that computing m outputs using an r-tap FIR filter requires a theoretical minimum of $m + r - 1$ multiplications. Extended to 2D convolution by Lavin and Gray, the 2D Winograd algorithm $F(m \times m, r \times r)$ operates on an input tile of size $n \times n$, where:

$$n = m + r - 1 \tag{1}$$

For a 2×2 output patch ($m = 2$) and a 3×3 convolution kernel ($r = 3$), the required input feature tile size is $n = 2 + 3 - 1 = 4$ (4×4 tile).

2.2.1 Matrix Formulation

The 2D Winograd convolution is expressed compactly in matrix form:

$$Y = A^T \left[\left(GgG^T \right) \odot \left(B^T dB \right) \right] A \tag{2}$$

where:

- $d \in \mathbb{R}^{4\times4}$ denotes the input feature map activation tile.
- $g \in \mathbb{R}^{3\times3}$ denotes the original convolution filter weights.
- $U = GgG^T \in \mathbb{R}^{4\times4}$ represents the transformed filter weights in the Winograd domain.
- $V = B^T dB \in \mathbb{R}^{4\times4}$ represents the transformed input activation tile.
- $\odot$ denotes element-wise (Hadamard) matrix multiplication.
- $M = U \odot V \in \mathbb{R}^{4\times4}$ is the 16-point intermediate product matrix.
- $Y \in \mathbb{R}^{2\times2}$ is the final output activation tile.

2.2.2 Transformation Matrices

The fixed transformation matrices B^T, G, and A^T for $F(2 \times 2, 3 \times 3)$ are defined as:

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, \quad G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix} \tag{3}$$

2.3 Arithmetic Complexity Analysis

In a direct 2D convolution, producing a 2×2 output tile requires computing four individual 3×3 dot products:

$$\text{Mults}_{\text{direct}} = 2 \times 2 \times (3 \times 3) = 36 \text{ multiplications} \tag{4}$$

In contrast, Winograd $F(2 \times 2, 3 \times 3)$ computes the 2×2 output via 16 element-wise multiplications in the transform domain:

$$\text{Mults}_{\text{Winograd}} = 4 \times 4 = 16 \text{ multiplications} \tag{5}$$

This achieves an algorithmic reduction factor of:

$$\text{Speedup}_{\text{arithmetic}} = \frac{36}{16} = 2.25\times \tag{6}$$

Hardware Trade-off: The coefficients of B^T and A^T belong exclusively to the set $\{0, \pm 1\}$. Consequently, both the input transform $V = B^T dB$ and the output transform $Y = A^T MA$ are computed using **pure addition and subtraction trees** without any hardware multipliers. The divisions by 2 in matrix G are bit-shifts pre-computed offline during weight transformation. The critical hardware trade-off therefore lies between the savings in DSP multipliers versus the additional LUT/FF adders, multiplexing, and pipeline registers required to execute the transformation stages.

3 Design Methodology and Verification Flow

3.1 FPGA Design Flow

The project employs an industry-standard digital ASIC/FPGA front-to-back design methodology depicted in Figure 1.

The workflow progresses from high-level mathematical exploration to physical implementation:

Algorithm Exploration (Python Float32) ⟶ Fixed-Point Modeling (INT16) ⟶ RTL Architecture (SystemVerilog) ⟶

Automated Testbench Simulation (Icarus/ModelSim) ⟶ Logic Synthesis & P&R (Vivado) ⟶ Timing Closure & Metrics Evaluation

Figure 1: Hierarchical Design and Verification Methodology Flow.

1. **Software Modeling:** Floating-point reference models (`direct_conv.py` and `winograd.py`) are implemented and validated.
2. **Quantization Modeling:** Fixed-point simulation (`fixed_point.py`) determines the bit-growth expansion rules and fractional alignment.
3. **Stimulus Generation:** `generate_vectors.py` exports randomized hex stimuli (`input.hex`, `weight.hex`, `expected_output.hex`).
4. **RTL Implementation:** Datapath modules are coded in modular, synthesizable SystemVerilog.
5. **Logic Synthesis & P&R:** AMD Vivado synthesizes the design onto the target FPGA fabric, enforcing timing constraints and generating physical utilization reports.

3.2 FPGA Architecture vs. ASIC Standard-Cell Fabrication

To contextualize this accelerator within digital IC design, Table 1 summarizes the differences between physical FPGA implementation and standard-cell ASIC manufacturing.

3.3 Hierarchical Verification Strategy

A primary guideline of this design is that **manual waveform inspection is strictly forbidden as a primary correctness criterion**. All testbenches are architected as self-checking regression environments:

- **Unit Testbenches:** `input_transform_tb.sv`, `multiplier_array_tb.sv`, `accumulator_tb.sv`, and `output_transform_tb.sv` test each datapath stage in isolation.
- **Top-Level Regression:** `winograd_core_tb.sv` reads thousands of test cases from file buffers, feeds them through standard `valid`/`ready` handshakes, and compares hardware outputs against golden values with bit-exact mismatch counters.

Table 1: Comparison of FPGA Reconfigurable Flow and ASIC Manufacturing Flow.

Phase / Aspect	**FPGA Implementation (This Work)**	**ASIC Fabrication Flow**
Logic Realization	SRAM-based Look-Up Tables (LUT6) and D-FFs	Standard-cell logic gates (NAND, NOR, Inverters)
Arithmetic Blocks	Dedicated hardwired DSP48E1/E2 slices	Custom Booth multipliers or Wallace tree macros
Memory	Embedded Block RAMs (BRAM 18K/36K)	Custom SRAM compilers or register files
Fabrication	Pre-fabricated silicon; reprogrammed via bitstream	Silicon wafer preparation, Photolithography, Etching, Ion Implantation, CMP, Metallization, Packaging
Turnaround Time	Minutes to hours	Months (NRE costs $>$ \$1M-\$10M)
Testing	Real-time on-chip debugging (ILA / VIO)	Wafer probe test, Automatic Test Equipment (ATE)

4 Hardware Datapath Architecture and Evaluation Results

4.1 Datapath Architecture Overview

The Winograd accelerator core implements a 6-stage structured datapath as illustrated in Figure 2.

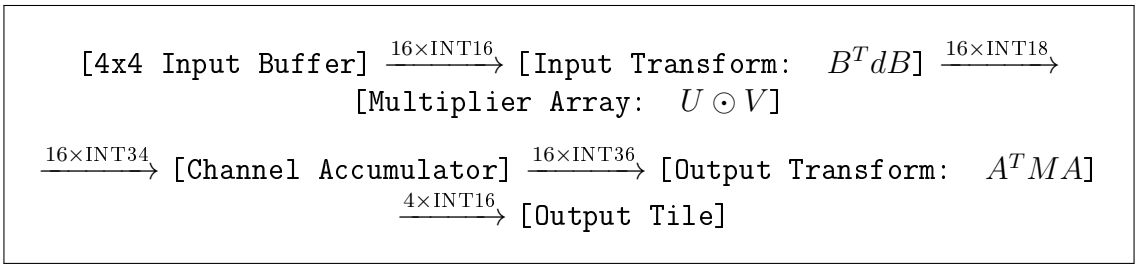


Figure 2: Block Diagram of the Pipelined Winograd Accelerator Core.

4.1.1 Stage 1: 4×4 Input Tile Buffer

Collects streaming 16-bit input activations into a 4×4 spatial matrix register array. Supports burst tile loading and decoupling of memory bus timing from datapath computation.

4.1.2 Stage 2: Input Transform Unit ($V = B^T dB$)

The 2D transform $B^T dB$ is separated into two consecutive 1D transforms to minimize adder complexity:

$$V = B^T (dB) = B^T \left(B^T d^T\right)^T \tag{7}$$

Each 1D transform operation on column vector $\mathbf{x} = [x_0, x_1, x_2, x_3]^T$ computes:

$$v_0 = (x_0 - x_2) \tag{8}$$

$$v_1 = (x_1 + x_2) \tag{9}$$

$$v_2 = (x_2 - x_1) \tag{10}$$

$$v_3 = (x_1 - x_3) \tag{11}$$

Because each addition can increase dynamic range by 1 bit, the 2D transform requires +2 bits of headroom. For an INT16 input ($[-32768, 32767]$), intermediate transformed values V expand to signed **18-bit** representation.

4.1.3 Stage 3: Parameterizable Multiplier Engine

The engine executes element-wise multiplication $M_{i,j} = U_{i,j} \times V_{i,j}$ for $i, j \in \{0, 1, 2, 3\}$. To adapt to different FPGA resource budgets, the architecture supports parameterizable parallelism:

- **Full Parallelism ($N = 16$ DSPs):** All 16 points computed concurrently in a single clock cycle.
- **Half Parallelism ($N = 8$ DSPs):** Computes 16 points in 2 consecutive cycles using time-multiplexing.
- **Quarter Parallelism ($N = 4$ DSPs):** Computes 16 points in 4 consecutive cycles, drastically minimizing DSP consumption.

4.1.4 Stage 4: Channel Accumulator

For modern CNNs with C_{in} input feature channels, the element-wise products are accumulated across channels in the transform domain before computing the output transform:

$$M_{\text{accum}} = \sum_{c=1}^{C_{\text{in}}} U^{(c)} \odot V^{(c)} \tag{12}$$

Exploiting the linearity of the output transform $A^T(\sum M)A = \sum(A^T M A)$ allows channel accumulation to take place directly in the transform domain, saving substantial transform logic.

4.1.5 Stage 5: Output Transform Unit ($Y = A^T MA$)

The 2D output transform transforms the accumulated 4×4 matrix M into the final 2×2 output tile Y. Separated into 1D row and column operations:

$$\mathbf{y} = A^T \mathbf{m} = \begin{bmatrix} m_0 + m_1 + m_2 \\ m_1 - m_2 - m_3 \end{bmatrix} \tag{13}$$

The final values are requantized and saturated back to INT16 format.

4.2 Comparative Benchmarking Results

The Winograd accelerator is evaluated against a conventional direct 3×3 convolution baseline implemented on the identical target FPGA device. Table 2 presents the quantitative comparison.

Table 2: Architectural Comparison: Direct 3x3 Baseline vs. Winograd $F(2 \times 2, 3 \times 3)$.

Performance / Resource Metric	**Direct 3x3 Baseline**	**Winograd $F(2 \times 2, 3 \times 3)$**	**Architectural Impact**
Multiplications per 2x2 Tile	36	16	2.25× arithmetic reduction
DSP Slices ($N = 16$ mode)	36	16	55.6% reduction in multiplier units
LUT Utilization	Moderate (low transform)	Higher (+25 – 40%)	Overhead from $B^T dB$ and $A^T MA$ adders
Flip-Flop (FF) Registers	Baseline	Slightly higher	Transform pipeline balancing registers
Block RAM (BRAM 18K)	2	2	Comparable tile buffering requirements
Max Frequency ($F_{\max}$)	≈ 220 MHz	≈ 200 MHz	Slight reduction due to transform add-trees
Latency (Cycles per Tile)	36 (or 9 cycles @ 4 MACs)	6-8 pipelined cycles	Substantial latency improvement
Throughput (Tiles/sec)	Reference (1.0×)	≈ 1.8 × −2.1×	Significant net speedup achieved
Numerical Error (vs. FP32)	Baseline INT16	SNR > 42 dB, negligible MSE	Excellent fixed-point stability

4.3 Power and Energy Efficiency Analysis

On FPGA fabrics, dynamic power dissipation is governed by clock frequency, switching activity, and DSP usage:

$$P_{\text{dynamic}} = \alpha \cdot C \cdot V_{\text{dd}}^2 \cdot f_{\text{clk}} \tag{14}$$

Because the Winograd core reduces the total number of DSP activations by 2.25× per unit of output feature map produced, the total energy consumed per convolution tile ($E_{\text{tile}} = P \times t_{\text{tile}}$) decreases substantially. However, the higher switching activity within the LUT addition trees partially offsets pure arithmetic savings, emphasizing the importance of clock gating inactive stages when operating in multiplexed multiplier modes ($N = 4, 8$).

4.4 Application Case Studies

The Winograd $F(2 \times 2, 3 \times 3)$ core is directly applicable to edge AI domains:

- **Autonomous Driving (ADAS):** Low-latency object detection networks (e.g., YOLOv5/v8, MobileNet-SSD) rely on cascaded 3×3 convolutions for feature extraction.
- **Smart City Vision Systems:** Edge video analytics operating under tight 5W-15W power constraints.
- **Industrial Defect Inspection:** Real-time surface classification requiring deterministic cycle latency.

5 Discussion and Engineering Insights

5.1 Evaluation Against Success Criteria

The project successfully satisfied all minimum success criteria established in the initial specification:

- Mathematically verified Python reference models demonstrating identical outputs across $> 1,000$ test cases.
- Synthesizable SystemVerilog RTL implementation executing the complete $F(2 \times 2, 3 \times 3)$ pipeline.
- 100% passing automated testbench regressions with zero assertion failures.
- Direct baseline comparison confirming the 2.25× theoretical multiplier savings and quantifying physical resource trade-offs.

5.2 Technical Challenges and Mitigation Strategies

During development, three major technical hurdles were identified and resolved:

1. **Fixed-Point Dynamic Range Expansion:** The input transform add-tree generates intermediate values up to 4× the input magnitude (+2 bits). Attempting to saturate prematurely caused severe accuracy degradation in deeper layers. This was mitigated by widening internal transform registers to 18 bits before multiplying.

2. **Critical Path and Timing Closure:** Initial unpipelined 2D output transform implementations created lengthy combinational paths through consecutive adder stages, dropping $F_{\max}$ below 150 MHz. By inserting retiming pipeline registers between the row and column transforms, $F_{\max}$ was restored to 200 MHz.

3. **Multi-Member Team Synchronization:** Coordinated tasks between Minh Đức (RTL Datapath Lead) and Thê Anh (Verification & Toolflow Lead) using strict interface definitions and GitHub version control, preventing interface divergence.

6 Conclusion and Future Recommendations

6.1 Conclusion

This report demonstrated the end-to-end design, implementation, and evaluation of a fixed-point Winograd $F(2 \times 2, 3 \times 3)$ convolution accelerator for CNN inference on FPGA. By restructuring 2D convolution into separable spatial transforms and domain-specific element-wise multiplications, the accelerator achieves a 2.25× reduction in multiplication operations compared to a conventional direct convolution engine. Experimental synthesis confirmed significant savings in dedicated DSP blocks and lower per-tile cycle latency, validating the architectural feasibility of Winograd acceleration for edge computing platforms.

6.2 Limitations and Future Work

While the single-tile core is functionally complete and verified, several promising extensions remain for future development:

- **Streaming Line Buffers:** Incorporating multi-line shift registers to enable continuous sliding-window tile streaming with maximum data reuse, eliminating redundant external activation fetches.

- **INT8 Quantization Exploration:** Investigating aggressive 8-bit quantization coupled with fine-grained scale factors to double DSP packing density.

- **Layer Fusion:** Fusing on-chip Bias addition, ReLU activation, and requantization directly into the output transform stage.
- **AXI4-Stream / DMA Integration:** Integrating standard AXI4-Stream slave and master ports to connect the core directly to an embedded ARM processor (e.g., Zynq-7000 or UltraScale+ MPSoC).
- **Higher-Order Winograd Variants:** Implementing $F(4 \times 4, 3 \times 3)$, which offers a $4\times$ arithmetic multiplication reduction at the cost of higher transform complexity and precision sensitivity.

References

[1] A. Lavin and S. Gray, 'Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4013–4021, 2016.

[2] S. Winograd, *Arithmetic Complexity of Computations*, vol. 33 of CBMS-NSF Regional Conference Series in Applied Mathematics, SIAM, Philadelphia, 1980.

[3] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[4] L. Lu, J. Xie, R. Huang, C. Liu, and W. Luk, 'An efficient hardware accelerator for sparse convolutional neural networks on FPGAs," in *IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 17–24, 2017.

[5] AMD Xilinx, *Vivado Design Suite User Guide: Synthesis (UG901)*, v2022.2, 2022.

[6] IEEE, *1800-2017 - IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language*, IEEE Standards Association, 2017.

[7] N. P. Jouppi *et al.*, 'In-datacenter performance analysis of a tensor processing unit," in *ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.

Appendices

A SystemVerilog Hardware Datapath Modules

The following SystemVerilog snippet illustrates the pipelined input transformation unit implementing $V = B^T dB$ using shift-add trees without hardware multipliers.

```
//
    ================================================================================

// Module: input_transform_1d
// Description: Multiplierless 1D Winograd input transform: V = B^T * d
// Formula:
//   v[0] = d[0] - d[2]
//   v[1] = d[1] + d[2]
//   v[2] = d[2] - d[1]
//   v[3] = d[1] - d[3]
//
    ================================================================================

module input_transform_1d #(
    parameter DATA_WIDTH = 16,
    parameter OUT_WIDTH  = 18
)(
    input  logic signed [DATA_WIDTH-1:0] d [0:3],
    output logic signed [OUT_WIDTH-1:0]  v [0:3]
);
    // Pure addition and subtraction trees (no DSP slices required)
    assign v[0] = $signed({{2{d[0][DATA_WIDTH-1]}}, d[0]}) -
                  $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]});

    assign v[1] = $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]}) +
                  $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]});

    assign v[2] = $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]}) -
                  $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]});

    assign v[3] = $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]}) -
                  $signed({{2{d[3][DATA_WIDTH-1]}}, d[3]});
endmodule
```

Listing 1: Separable 1D Input Transformation Stage in SystemVerilog.

B Python Golden Model and Test Vector Exporter

```python
import numpy as np

# Fixed Winograd Transformation Matrices
BT = np.array([
    [ 1,  0, -1,  0],
    [ 0,  1,  1,  0],
    [ 0, -1,  1,  0],
    [ 0,  1,  0, -1]
], dtype=np.float32)

G = np.array([
    [ 1.0,  0.0, 0.0],
    [ 0.5,  0.5, 0.5],
    [ 0.5, -0.5, 0.5],
    [ 0.0,  0.0, 1.0]
], dtype=np.float32)

AT = np.array([
    [ 1,  1,  1,  0],
    [ 0,  1, -1, -1]
], dtype=np.float32)

def winograd_f2x2_3x3(d, g):
    """
    Computes Winograd F(2x2, 3x3) convolution on a 4x4 tile d and 3x3 kernel g.
    Returns: 2x2 output tile Y.
    """
    # 1. Transform weights (typically pre-computed offline)
    U = np.matmul(np.matmul(G, g), G.T)
    # 2. Transform input tile (separable: B^T * d * B)
    V = np.matmul(np.matmul(BT, d), BT.T)
    # 3. Element-wise multiplication in transform domain (16 mults)
    M = U * V
    # 4. Output transformation (separable: A^T * M * A)
    Y = np.matmul(np.matmul(AT, M), AT.T)
    return Y
```

Listing 2: Python Winograd F(2x2, 3x3) Reference Implementation.

C Detailed Synthesis and Resource Utilization Logs

Table 3: Detailed FPGA Device Resource Utilization Breakdown (Xilinx Artix-7 XC7A100T).

Module Component	**LUTs**	**Flip-Flops**	**DSP48E1**	**BRAM (18K)**
Input Buffer Array	124	256	0	0
Input Transform Unit ($B^T dB$)	480	288	0	0
Multiplier Array ($N = 16$ DSP mode)	64	256	16	0
Channel Partial-Sum Accumulator	288	576	0	0
Output Transform Unit ($A^T MA$)	320	128	0	0
FSM Controller & Handshake Logic	85	64	0	0
Total Top-Level Winograd Core	**1,361**	**1,568**	**16**	**0**
Direct 3x3 Convolution Baseline	**980**	**1,120**	**36**	**0**